

Shipping to Production

Approaches for shipping code to production reliably, every time.



GERGELY OROSZ

MAY 24, 2022 • PAID



149



19



Share



👋 Hi, this is [Gergely](#) with a free issue of the Pragmatic Engineer Newsletter. If you're not a full subscriber yet, you missed the second part of [Inside Facebook's engineering culture](#) and the one [The tech boom potentially being over](#). Subscribe to get this newsletter every week 📌

Q: As an engineer at a fast-growing startup, I'd like to learn more about how small and large companies ship code to production. Are there any 'best practices' worth following, or does everyone come up with their own approach?

Great question. How you ship your code to production in a way that is fast and reliable, is a question more engineers and engineering leaders should educate themselves on. The teams and companies that can do both – ship quickly / frequently and with good quality – have a big advantage over competitors who struggle with either constraint.

In this issue we cover:

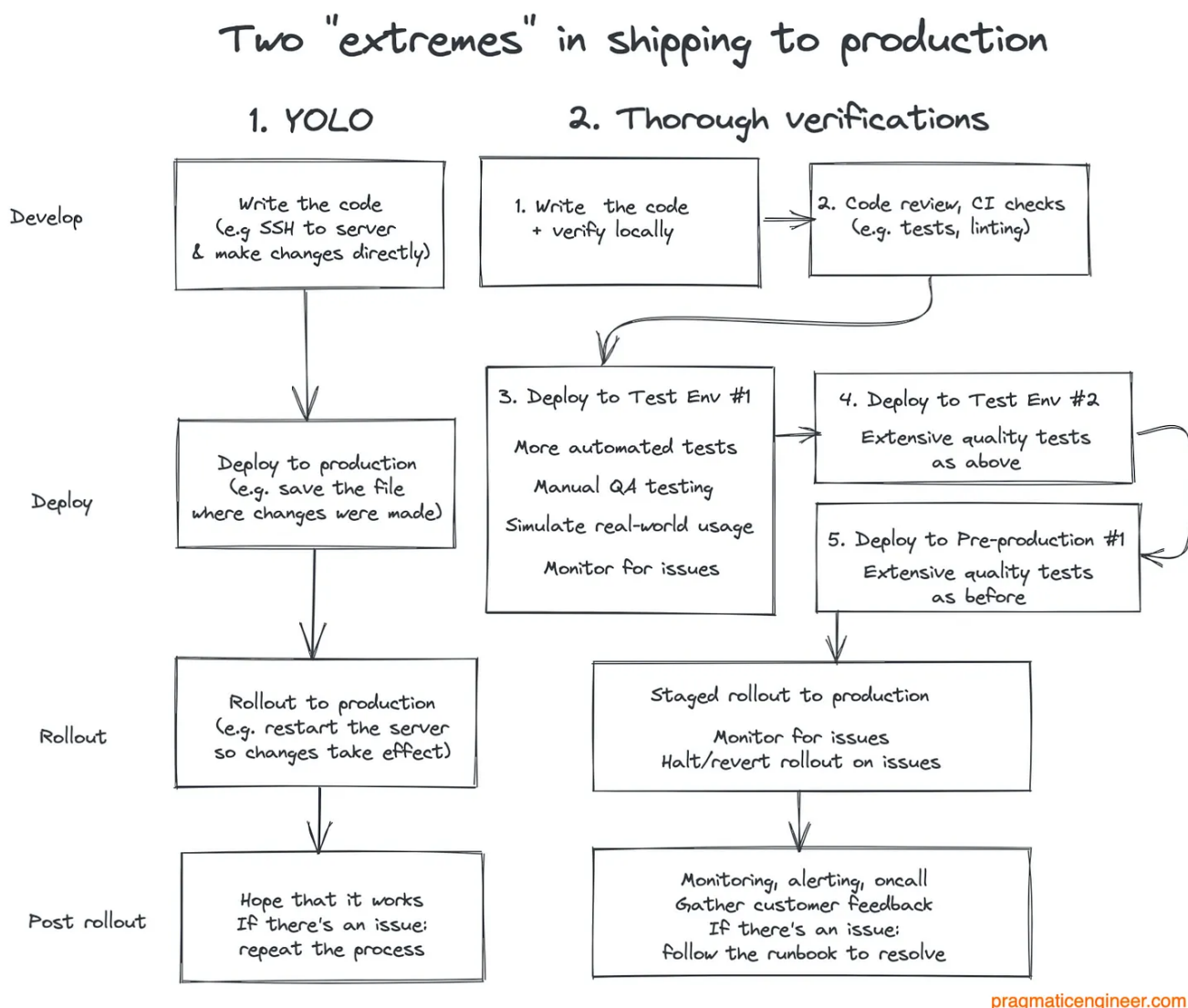
1. The extremes of shipping to production.
2. Typical processes at different types of companies.
3. Principles and tools for shipping to production responsibly.
4. Additional verification layers and advanced tools.
5. Taking pragmatic risks to move faster.
6. Deciding which approach to take.

7. Other things to incorporate into the deployment process.

Read the free, slightly shorter version of this article on [The Pragmatic Engineer Blog: Shipping to Production](#).

1. The Extremes of Shipping to Production

When it comes to shipping code, it's good to understand the two extremes of how code can make it to users. The table below shows the 'thorough' way and the 'YOLO' way.



Two "extremes" of shipping to production: YOLO and thorough verification

1. YOLO shipping - YOLO stands for “you only live once.” This is the approach many prototypes, side projects, and unstable products like alpha or beta versions of products, use. In some cases, it’s also how urgent changes make it into production.

The idea is simple: make the change in production, then see if it works – all in production. Examples of YOLO shipping include:

- SSH into a production server → open an editor (e.g. vim) → make a change in a file → save the file and / or restart the server → see if the change worked.
- Make a change to a source code file → force land this change without a code review → push a new deployment of a service.
- Log on to the production database → execute a production query fixing a data issue (e.g. modifying records which have issues) → hope this change has fixed the problem.

YOLO shipping is as fast as it gets in terms of shipping a change to production. However, it also has the highest likelihood of introducing new issues into production, as this type of shipping has no safety nets in place. With products that have little to no production users, the damage done by introducing bugs into production can be low, and so this approach is justifiable.

YOLO releases are common with:

- Side projects.
- Early-stage startups with no customers.
- Mid-sized companies with poor engineering practices.
- When resolving urgent incidents at places without well-defined incident handling practices.

As a software product grows and more customers rely on it, code changes need to go through extra validation before reaching production. Let’s go to the other extreme: a team obsessed with doing everything possible to ship close to zero bugs into production.

2. Thorough verification through multiple stages is where a mature product with many valuable customers tends to end up, when a single bug may cause major problems. For example, if bugs result in customers losing money, or customers leaving for a competitor, this rigorous approach might be used.

With this approach, several verification layers are in place, with the goal of simulating the real-world environment with ever more accuracy. Some layers might be:

1. **Local validation.** Tooling for software engineers to catch obvious issues.
2. **CI validation.** Automated tests like unit tests and linting running on every pull request.
3. **Automation before deploying to a test environment.** More expensive tests such as integration tests or end-to-end tests run before deployment to the next environment.
4. **Test environment #1.** More automated testing, like smoke tests. Quality assurance engineers might manually exercise the product, running both manual tests and doing exploratory testing.
5. **Test environment #2.** An environment where a subset of real users – such as internal company users or paid beta testers – exercise the product. The environment is coupled with monitoring and upon a sign of regression, the rollout is halted.
6. **Pre-production environment #3.** An environment where the final set of validations are run. This often means running another set of automated and manual tests.
7. **Staged rollout.** A small subset of users get the changes, and the team monitors for key metrics to remain healthy and checks customer feedback. The staged rollout strategy is orchestrated based on the riskiness of the change being made.
8. **Full rollout.** As the staged rollout increases, at one point, changes are pushed to all customers.
9. **Post-rollout.** Issues will come up in production, and the team has monitoring and alerting set up, and a feedback loop with customers. If an issue arises, it is dealt

with by following a standard process. Once an outage is resolved, the team follows [incident review and postmortem best practices, which we've covered before](#).

Such a heavyweight release process can be seen at places like:

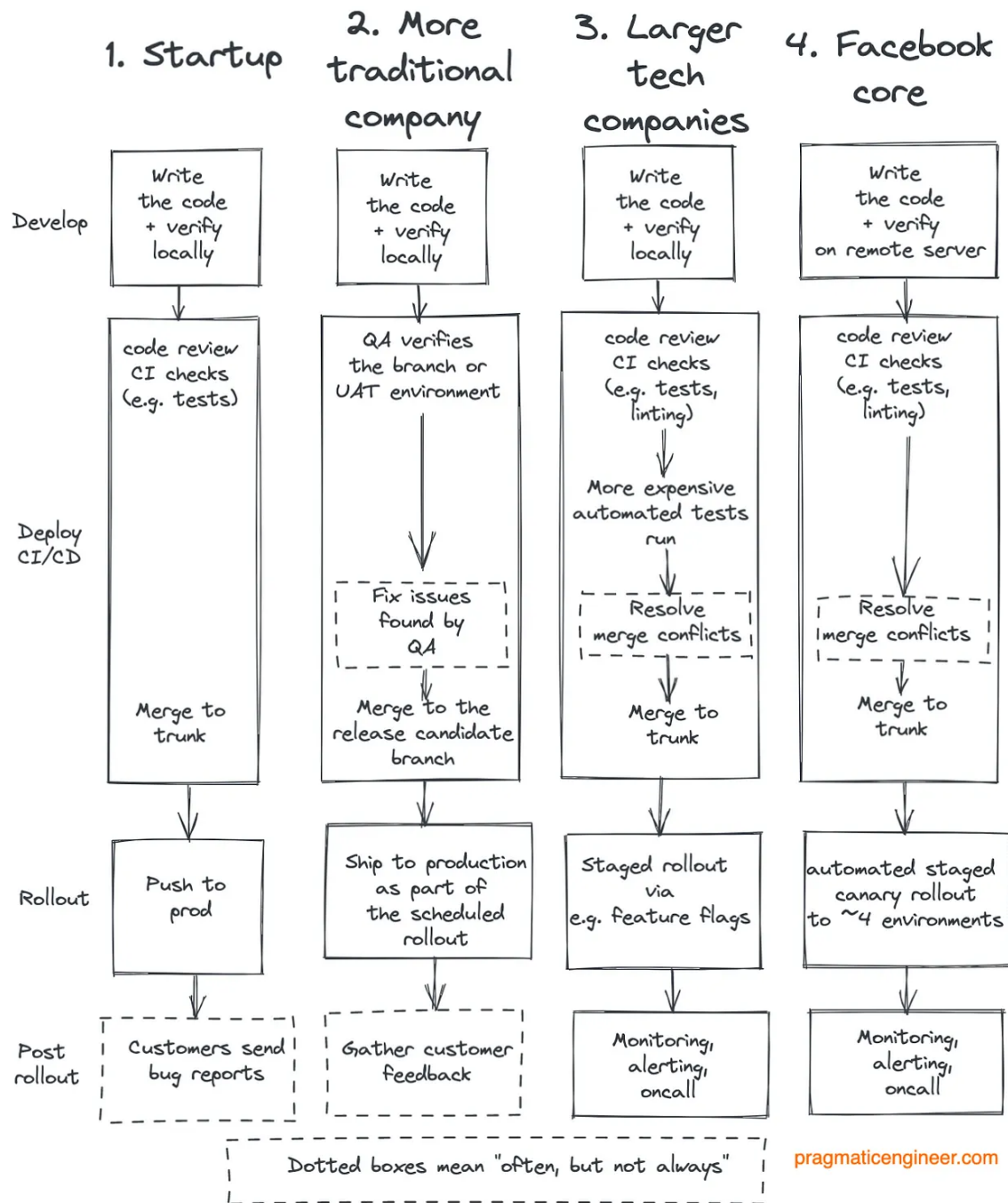
- Highly regulated industries, such as healthcare.
- Telecommunications providers, where it is not uncommon to have ~6 months of thorough testing of changes, before shipping major changes to customers.
- Banks, where bugs could result in significant financial losses.
- Traditional companies with legacy codebases that have little automated testing. However, these companies want to keep the quality high and are happy slowing down releases by adding more verification stages.

2. Typical processes at different types of companies

What are "typical" ways to ship code into production at different types of companies? Here's my attempt to generalize these approaches, based on observation. The below is a generalization, as not every company's practices will match these processes. Still, I hope it illustrates some differences in how different companies approach shipping to production:

How do various companies typically ship to production?

An admittedly imperfect attempt to visualize the differences in approaches



Different ways various companies typically ship to production.

Notes on this diagram:

1. Startups: Typically do fewer quality checks than other companies.

Startups tend to prioritize moving fast and iterating quickly, and often do so without much of a safety net. This makes perfect sense if they don't – yet – have customers. As the company attracts users, these teams need to start to find ways to not cause regressions or ship bugs. They then have the choice of going down one of two paths: hire QAs or invest in automation.

2. Traditional companies: Tend to rely more heavily on QAs teams.

While automation is sometimes present in more traditional companies, it's very typical that they rely on large QA teams to verify what they build. Working on branches is also common; it's rare to have trunk-based development in these environments.

Code mostly gets pushed into production on a schedule, for example weekly or even less frequently, after the QA team has verified functionality.

Staging and UAT (User Acceptance Testing) environments are more common, as are larger, batched changes shipped between environments. Sign-offs from either the QA team or the product manager – or project manager – are often required to progress the release from one stage to the next.

3. Large tech companies: Typically invest heavily in infrastructure and automation related to shipping with confidence.

These investments often include automated tests running quickly and delivering rapid feedback, canarying, feature flags and staged rollouts.

These companies aim to keep a high quality bar, but also to ship immediately when quality checks are complete, working on trunk. Tooling to deal with merge conflicts becomes important, given some of these companies can see over 100 changes on trunk per day.

4. Facebook core: Has a sophisticated and effective approach few other companies possess.

Facebook's core product is an interesting one. It has fewer automated tests than many would assume, but, on the other hand, it has an exceptional automated canarying

functionality, where the code is rolled out through 4 environments: from a testing environment with automation, through one that all employees use, through a test market of a smaller region, to all users. In every stage, if the metrics are off, the rollout automatically halts.

From the article [Inside Facebook's engineering culture](#), in which a former Facebook employee relates how an Ads issue was caught before causing an outage:

"I was working in Ads and I remember this particular bad deploy which would have made a massive financial loss to the company. The code change broke a specific ad type.

'This issue was caught in stage 2 out of 4 stages. Each stage had so many health metrics monitored that if something went wrong, these metrics would almost always trigger - like this first safety net had done.

"All Facebook employees used the before-production builds of the Facebook website and apps. This means that Facebook has a massive testing group of ~75,000 people testing their products before they go into production, and flagging any degradations."

3. Principles and tools for responsibly shipping to production

What are principles and approaches worth following if you are to ship changes to production in a responsible manner? Here are my thoughts. Note that I am not saying you need to follow all the ideas below. But it's a good exercise to question why you should *not* implement any of these steps.

1. Have a local or isolated development environment. Engineers should be able to make changes either on their local machine, or in an isolated environment unique to them. But it's more common to have developers work in local environments. However, places like Meta are moving over to working on remote servers dedicated to each engineer. From the article [Inside Facebook's Engineering culture](#):

"Most developers work with a remote server, not locally. Starting from around 2019, all web and backend development is done remotely, with no code copied locally, and Nuclide facilitating this workflow. In the background, Nuclide was using virtual machines (VMs) at first, later moving to OnDemand instances – similar to how GitHub Codespaces works today – years before GitHub launched Codespaces.

"Mobile development is still mostly done on local machines, as doing this in a remote setup as with web and backend has tooling challenges."

2. Verify locally. After writing the code, do a local test to make sure that it works as it should.

3. Consider edge cases and test for them. Which non-obvious cases does your code change need to account for? What are use cases that exist in the real world that you might have not accounted for?

Before finalizing work on the change, put together a list of these edge cases. Consider writing automated tests for these, if possible. At least do manual testing on them.

Coming up with a list of edge cases is a scenario when QA engineers or testers can be very helpful in coming up with non-conventional edge cases, if you work with them.

4. Write automated tests to validate your changes. After you manually verify your changes, exercise your changes with automated tests. If following a methodology like TDD, you might do this the other way around by writing automated tests first, then ensuring those tests are passed following your changes.

5. Get another pair of eyes to look at it: code review. With all changes validated, get someone else with context to look at your code changes. Write a clear and concise description of your changes, make it clear which edge cases you've tested and get a code review. *Read more about [how to do good code reviews and better code reviews](#).*

6. All automated tests pass, minimizing the chances of regressions. Before pushing the code, run all the existing tests for the codebase. This is typically done automatically, via the CI / CD system (continuous integration / continuous deployment).

7. Have monitoring in place for key product characteristics related to your change. How will you know if your change breaks things that automated tests don't check for? Unless you have ways to monitor health indicators on the system, you won't. So make sure there are health indicators written for the change, or other ones you can use.

For example at Uber, most code changes were rolled out as experiments with a defined set of metrics they were expected to either not impact, or to improve. One of the metrics that was expected to remain unchanged was the percentage of people successfully taking Uber trips. If this metric dropped with a code change, an alert was fired and the team making the code change needed to investigate if their change had done something to degrade the user experience.

8. Have oncall in place, with enough context to know what to do if things go wrong. After the change is shipped to production, there's a fair chance some defects will only become visible much later. That why's it good to have an oncall rotation in place with engineers who can respond to either health alerts, or to inbound from customers, or customer support.

Make sure the oncall is organized in a way that those performing oncall duty have enough context on how to mitigate outages. In most cases, teams have runbooks with sufficient details for confirming outages and mitigating them. Many teams also have oncall training, and some do oncall situation simulations to prepare team members for their oncall.

9. Operate a culture of blameless incident handling, where the team learns and improves following incidents. I cover this topic in-depth in the article [Incident review and postmortem best practices](#).

4. Additional verification layers and approaches

What are layers, tools and approaches that some companies use as additional ways to deliver reliable code to production? It's common for teams to use some of the approaches below, but rarely all of them at once. Some approaches cancel each other

out; for example, there's little reason to have multiple testing environments if you have multi-tenancy in place and are already testing in production.

Here are 9 approaches that provide additional safety nets:

1. Separate deployment environments. Setting up separate environments to test code changes is a common way to add an additional safety net to the release process. Before code hits production, it's deployed to one of these environments. These environments might be called testing, UAT (user acceptance testing), staging, pre-prod (pre-production) and others.

For companies with QA teams, QA often exercises a change in this environment, and looks for regressions. Some environments might be built for executing automated tests, such as end-to-end tests, smoke tests or load tests.

While these environments look good on paper, they come with heavy maintenance costs. This is both in resources, as machines need to be operated so this environment is available, and even more so in keeping the data up-to-date. These environments need to be seeded with data that is either generated, or brought over from production.

Read more about deployment and staging environments in the article [Building and releasing to deployment environments](#), published by the Harness team.

2. Dynamically spin up testing / deployment environments. Maintaining deployment environments tends to create a lot of overhead. This is especially true when, for example, [doing data migrations](#) means data needs to be updated in all the test environments.

For a better development experience, invest in automation to spin up test environments, including the seeding of the data they contain. Once this is in place, it opens up opportunities to do more efficient automated testing, for people to validate their changes more easily, and to create automation that better fits your use cases.

3. A dedicated QA team. An investment many companies make which want to reduce defects, is hiring a QA team. This QA team is usually responsible for manual and exploratory testing of the product.

My view is if you have a QA team that does only manual testing, then there is value in this. However, if the team executes manual tests on things that could be automated, the time overhead of manual testing will slow the team down instead of speeding them up.

In productive teams, I've observed QA transition into domain experts and help engineers anticipate edge cases. They also do exploratory testing and find more edge cases and unexpected behaviors.

However in these productive teams, QA also become QA engineers, not manual testers. They start to get involved in the automating of tests, and have a say in shaping the automation strategy so it speeds up the process of code changes making it into production.

4. Exploratory testing. Most engineers are good at testing the changes they make, verifying they work as expected, and at considering edge cases. But what about testing in a way which relates to how retail users of the system utilize the product?

This is where exploratory testing comes in.

Exploratory testing is trying to simulate how customers might use the product, in order to come across the edge cases they may stumble upon. Good exploratory testing requires empathy with users, an understanding of the product, and tooling with which to simulate various use cases.

At companies with dedicated QA teams, these QA teams usually do exploratory testing. At places without dedicated QA teams, it's either down to engineers to perform this testing, or companies will sometimes contract vendors specializing in exploratory testing.

5. Canarying. Canarying comes from the phrase "canary in the coal mine." In the early 20th century, miners took a caged canary bird with them down a mine. This bird has a lower tolerance for toxic gasses than humans, so if the bird stopped chirping or fainted, it was a warning sign to the miners that gas was present and they evacuated.

Today, canary testing means rolling out code changes to a smaller percentage of the user base, then monitoring the health signals of this deployment for signs that something's not right. A common way to implement canarying is to either route traffic to the new version of the code using a load balancer, or to deploy a new version of the code to a single node.

Read more about canarying [in this article](#) by the LaunchDarkly team.

6. Feature flags and experimentation. Another way to control the rollout of a change is to hide it behind a feature flag in the code. This feature flag can then be enabled for a subset of users, and those users execute the new version of the code.

Feature flags are easy enough to implement in the code, and might look something like this flag for an imaginary feature called 'Zeno':

```
if( featureFlags.isEnabled("Zeno_Feature_Flag")) {  
  // New code to execute  
} else {  
  // Old code to execute  
}
```

Feature flags are a common way to allow running experiments. An experiment means bucketing users in two groups: a treatment group (the experiment) and the control group (those not getting the experiment). These two groups get two different experiences, and the engineering and data science team evaluate and compare results.

7. Staged rollout. Staged rollouts mean shipping changes step by step, evaluating the results at each stage, and then proceeding further. Staged rollouts typically define either the percentage of the user base to get the changed functionality, or the region where this functionality should roll out, or a mix of both.

A staged rollout plan may look like this:

- Phase 1: 10% rollout in New Zealand (a small market to validate changes)
- Phase 2: 50% rollout in New Zealand

- Phase 3: 100% rollout in New Zealand
- Phase 4: 10% rollout, globally
- Phase 5: 25% rollout, globally
- Phase 6: 50% rollout, globally
- Phase 7: 100% rollout, globally

Between each rollout stage, a criteria is set for when the rollout can continue. This is typically defined as there being no unexpected regressions and the expected changes (or absence thereof) to business metrics being observed.

8. Multi-tenancies. An approach that is growing in popularity is using production as the one and only environment to deploy code to, including testing in production.

While testing in production sounds reckless, it's not if done with a multi-tenant approach. Uber describes its journey from a [staging environment, through a test sandbox with shadow traffic, all the way to tenancy-based routing](#).

The idea behind tenancies is the tenancy context is propagated with requests. Services getting a request can tell if this request is for a production request, for a test tenancy, for a beta tenancy, and so on. Services have logic built in to support tenancies, and might process or route requests differently. For example, a payments system getting a request with a test tenancy would likely mock the payment, instead of making an actual payment request.

For more details on this approach, read [how Uber implemented a multi-tenancy approach](#), and also [how Doordash did](#).

8. Automated rollbacks. A powerful way to increase reliability is to make rollbacks automatic for any code changes that are suspected of breaking something. This is an approach that Booking.com uses, among others. Any experiment that's found to be degrading key metrics is shut down and the change rolled back:



Stefano Baccianella
@mangiucugna

@GergelyOrosz Not sure if still the case but I presume so. Booking.com's experiment tool has a killswitch that will immediately kill an experiment in production if any of the health metrics (i.e. errors) spikes up right after an experiment is started.



booking.com

Booking.com: The largest selection of hotels, homes, and vacation rentals

12:33 PM • May 23, 2022

19 Likes 1 Retweet

GitHub Distinguished Engineer Jaana Dogan makes a similar case for automatic rollbacks coupled with staged rollouts:



Jaana Dogan ヤナ ドガン

@rakyll

One more detail is what type of investment that goes into auto rollbacks at some companies vs others. When you have solid multi staged auto rollouts and auto rollbacks in place, you rarely fear breaking production.



Gergely Orosz @GergelyOrosz

Shipping to production: how do different types of companies typically approach this?

Here's an attempt to visualize the differences:

- Startups usually do few quality checks & move fast
- Traditional companies tend to rely heavier on QAs
- Large tech companies invest in infra: <https://t.co/Z0wvf0gIMT>

8:19 PM • May 19, 2022

259 Likes 30 Retweets

9. Automated rollouts and rollbacks across several test environments. Taking automated rollbacks a step further, and combining them with staged rollouts and multiple testing environments, is an approach that Facebook has uniquely implemented for their core product.

5. Taking pragmatic risks to move faster

There are occasions when you want to move faster than normal, and are comfortable taking a bit more risk to do so. What are pragmatic practices for this?

Decide which process or tool it is *not okay* to ever bypass. Is force-landing without running any tests ever an option? Can you make a change to the codebase without anyone else looking at it? Can a production database be changed without testing?

It's down to every team – or company – to decide which processes cannot be bypassed. For all the above examples, if this question comes up at a mature company with a large number of users who depend on their products, I'd think carefully before breaking rules, as this could cause more harm than good. If you do decide to bypass rules in order to move faster, then I recommend getting support from other people on your team before embarking upon this course of action.

Give a heads up to relevant stakeholders when shipping “risky” changes. Every now and then you'll ship a change that is more risky and less tested than you'd ideally want. It's good practice to give a heads up to the people who might be able to alert you if they see something strange unfolding. Stakeholders worth notifying in these cases can include:

- Members of your team.
- Oncalls for teams that depend on your team, and ones which you depend on.
- Customer-facing stakeholders whom users contact.
- Customer support.

- Business stakeholders who have access to business metrics and can notify you if they see something trending in the wrong direction.

Have a rollback plan that is easy to execute. How can you revert a change which causes an issue? Even if you're moving fast, make sure you have a plan that's easy enough to execute. This is especially important for data changes and configuration changes.

Revert plans used to be commonly added to diffs in the early days of Facebook. From [Inside Facebook's Engineering Culture](#):

"Early engineers shared how people used to also add a revert plan to their diff to instruct how to undo the change, in the frequent case this needed to be done. This approach has improved over the years with better test tooling."

Tap into customer feedback after shipping risky changes. Get access to customer feedback channels like forums, customer reviews, and customer support after you ship a risky change. Proactively look through these channels to see if any customers have issues stemming from the change you rolled out.

Keep track of incidents, and measure their impact. Do you know how many outages your product had during the past month? The past three months? What did customers experience? What was the business impact?

If the answer to these questions is that you don't know, then you're flying blind and don't know how reliable your systems are. Consider changing your approach to track and measure outages, and accumulate their impacts. You need this data to know when to tweak your release processes for more reliable releases. You'll also need it if you are to use error budgets.

Use error budgets to decide if you can do "risky" deployments. Start measuring the availability of your system with measurements like SLIs (Service Level Indicators) and SLOs (Service Level Objectives,) or by measuring how long the system is degraded or is down.

Next, define an [error budget](#), an amount of service degradation that you deem acceptable for users, temporarily. For as long as this error budget is not exceeded, more risky deployments – those more likely to break the service – might be fine to go ahead with. However, once you hit this quota, take no further shortcuts.

6. Deciding on the approach you take

We've covered a lot of ground and many potential approaches for shipping reliably to production, every time. So how do you decide which approaches you take? There are a few things to consider.

How much are you willing to invest in modern tooling, in order to ship with more iterations? Before getting into the tradeoffs of various approaches, you need to be honest with yourself on how much investment you, your team, or your company are willing to make in tooling.

Many of the approaches we have gone through involve putting tooling in place; most of which can be integrated through vendors, but some of them you need to buy. If you work at a company with [platform teams](#) or SRE teams focused on reliability, then you might have lots of support, already. If working at a smaller company, you might need to make the case for investing in tooling.

How big an error budget can your business realistically afford? If a bug makes it to production for a few customers, what is the impact? Does the business lose millions of dollars, or are customers mildly annoyed – but not churning – so long as the bug is fixed quickly?

For businesses like private banks, a bug in the money flows can mean massive losses. For businesses like Facebook, a UI bug that is fixed quickly will not have much impact. This is why Facebook [has less testing in place than many other Big Tech companies](#), and why there [is no QA function within the company](#).

How much legacy infrastructure and code do you need to work with? It can be expensive, time-consuming and difficult to modernize legacy systems with some modern practices like automated testing, staged rollouts or automatic rollbacks.

Do an inventory on what the existing tech stack is, to evaluate if it is worth modernizing or not. There will be cases where there is little point investing in modernization.

What iteration speed should you target as a minimum? The faster engineers can ship their code to production, the faster they get feedback. In many cases, faster iteration speed results in higher quality because engineers push smaller and less risky changes to production.

According to the DORA metrics (DevOps Research and Assessment metrics), elite performers do [on-demand, multiple deployments per day](#). The lead time for changes – the time between code committed to it reaching production – is less than a day for elite performers. Although I am not the biggest fan of DORA metrics – as I think they don't paint a full picture of engineering excellence, and focusing only on those numbers can be misleading – these observations on how nimble teams ship to production quickly, do match my own.

At most Big Tech firms and at many high-growth startups it takes less than a day – typically a few hours – from code being committed to it reaching production, and teams deploy on-demand, multiple times per day.

If you have QA teams, what is the end goal of the QA function? QA teams are typical at companies that either cannot afford many bugs in production, or where they lack the capability to automate testing, so make up for it by hiring QA specialists.

Still, I suggest setting a goal for what the QA organization should evolve into, and how it should support engineering. If all goes well, what will QA look like in a few years' time? Will they do only manual testing? Surely not. Will they own the automation strategy? Or help engineering teams ship code changes to a deployment environment in the same day? Allow for engineers to ship to production in less than a week?

Think ahead and set goals that result in shorter iteration, faster feedback loops, and catching and fixing issues more quickly than today.

Consider if you will invest in advanced capabilities. As of today, some deployment capabilities are still considered less common, as they are challenging to build. These include:

- Sophisticated monitoring and alerting setups, where code changes can easily be paired with monitoring and alerting for key system metrics. In such environments, engineers can easily monitor whether their changes regress system health indicators.
- Automated staged rollouts, with automated rollbacks.
- The ability to generate dynamic testing environments.
- Robust integration, end-to-end and load testing capabilities.
- Testing in production through multi-tenancy approaches.

Decide if you want – or need – to invest in any of these complex approaches, which could result in shipping even faster, with more confidence.

7. Other things to incorporate into the deployment process

In this article, I've skipped past other parts of the release process to production that any mature product and company will need to address. While we haven't gone into detail in this article, don't neglect them. They include:

- **Security practices.** Who is allowed to make changes to systems? How are these changes logged for a later audit? How are security audits done on code changes, to reduce the likelihood of vulnerabilities making it into the system? Which secure coding practices are followed, and how are these encouraged or enforced?
- **Configuration management.** Many changes to systems are configuration changes. How is configuration stored? How are changes to configurations signed off and tracked?
- **Roles and responsibilities.** Which roles are in the release process? For example, who owns the deployment systems? In the case of batched deployments, who owns the following up on issues, and giving the green light to deployments?
- **Regulation.** When working in highly regulated sectors, shipping changes might include working with regulators and adhering to rules that are more heavyweight, and deliberately slow down the pace of shipping to customers. Regulatory

requirements could include legislation like GDPR (General Data Protection Regulation), PCI DSS (Payment Card Industry Data Security Standard), HIPAA (Health Insurance Portability and Accountability Act, FERPA (Family Educational Rights and Privacy Act,) FCRA (Fair Credit Reporting Act,) Section 508 when working with US federal agencies, European Accessibility Act when developing for a government within the EU, or country-specific privacy laws, regulations and more.

Takeaways

I suggest that one of the biggest takeaways is to not *only* focus on how you ship to production, but how you can revert mistakes when they happen. A few other closing thoughts:

- **Shipping quickly vs doing it with high quality; you can have both!** One misconception some people have is thinking this is a zero-sum choice; either you ship quickly, or you have a thorough but slow process of high quality. On the contrary; many tools help you to ship quickly with high quality, enabling you to have both. Code reviews, CI / CD systems, linting, automatically updated testing environments or test tenancies can all help you to ship rapidly, with high quality.
- **Know the tools at your disposal.** This article lists many different approaches you can use to ship faster, with more confidence.
- **QA is your friend in helping build a faster release process.** While some engineers working with QA engineers and testers might feel that QA slows things down, I challenge this notion. The goal of QA is to release high-quality software. It is also their goal to not get in the way of shipping quickly. If you work with QAs, collaborate with them on how you can improve the release process to make it faster, without reducing quality.
- **It's harder to retrofit legacy codebases with tools that enable faster shipping.** If you are starting a product from scratch, it's easier to set up practices that allow for rapid shipping, than it is to retrofit them to existing codebases. This is not to say you cannot retrofit these approaches, but doing so will be more expensive.

- **Accept you will ship bugs.** Focus on how quickly you can fix them! Many teams obsess far too much on how to ship close to zero bugs to production, at the expense of thinking about how to quickly spot issues and resolve them promptly.



Gergely Orosz
@GergelyOrosz

On software quality: many teams obsess far too much on how to ship with close to no bugs to production:

but not enough about how to quickly spot these, and resolve them rapidly - doing this either automatically, or with a simple step.

Resilient products/platforms have both.

11:45 AM · May 23, 2022

995 Likes 131 Retweets

What is your experience of shipping to production? Which approaches worked well in your environment, and why? Leave a comment below.

Further reading

Several companies have shared their approach to shipping code into production. Below are a few articles worth reading to learn more about different tactics:

- Monzo: [deploying to production over 100 times a day](#) (2022)
- Wayfair: [increasing deployment frequency 5x in 2 years](#) (2021)
- Funda: [release automation](#) (2021)
- Harness: [building & releasing to deployment environments](#) (2021)
- LinkedIn: [learnings on continuous deployment](#) (2019)
- Honeycomb CTO Charity Majors: [I test in production](#) (2019)
- Up Bank: [continuous delivery](#) (2018)
- Facebook: [rapid release at massive scale](#) (2017)

- Uber: [deploying to daily, with confidence](#) (2016)
- The Guardian: [adopting continuous delivery](#) (2015)

How did you like this week's newsletter? 🤔

[Amazing](#) • [Great](#) • [Good](#) • [OK](#) • [So-so](#)

If you've enjoyed this newsletter, subscribe to get weekly issues. In every issue, I cover challenges at big tech and high-growth startups through the lens of engineering managers and senior engineers. 📌



149 Likes

19 Comments



Write a comment...



Daniel Schniepp May 24, 2022 · edited May 24, 2022 ❤️ Liked by Gergely Orosz

Thank you for this great write-up. Actually I was looking into that topic over the last weeks. I would be curious how Stripe, Monzo or Bunq deploy in detail. How do they use staging, and for how long are features on staging before they get released to production? Because I see the danger of features idling on staging environments for days before they come into production. Would be super interested in a detailed write-up for a regulated company with a high velocity.

❤️ LIKE (3) 💬 REPLY ...

3 replies by Gergely Orosz and others



Yair Cohen Jun 12, 2022 ❤️ Liked by Gergely Orosz

Really enjoyed this one, thanks Gergely.

♡ LIKE (1) 💬 REPLY ...

17 more comments...

© 2023 Gergely Orosz · [Privacy](#) · [Terms](#) · [Collection notice](#)
[Substack](#) is the home for great writing